

Movement Component Architecture

Executive summary

Snake's movement logic currently lives directly on `SnakeSegment` / `SnakeHead` — each segment carries its own `direction`, `pendingDirection`, `moveTimer`, and `moveInterval` fields, and `SnakeHead` binds input directly to itself. This makes movement a Snake-specific, non-reusable trait: nothing else in the engine could "just become movable" without duplicating that state and logic onto its own class.

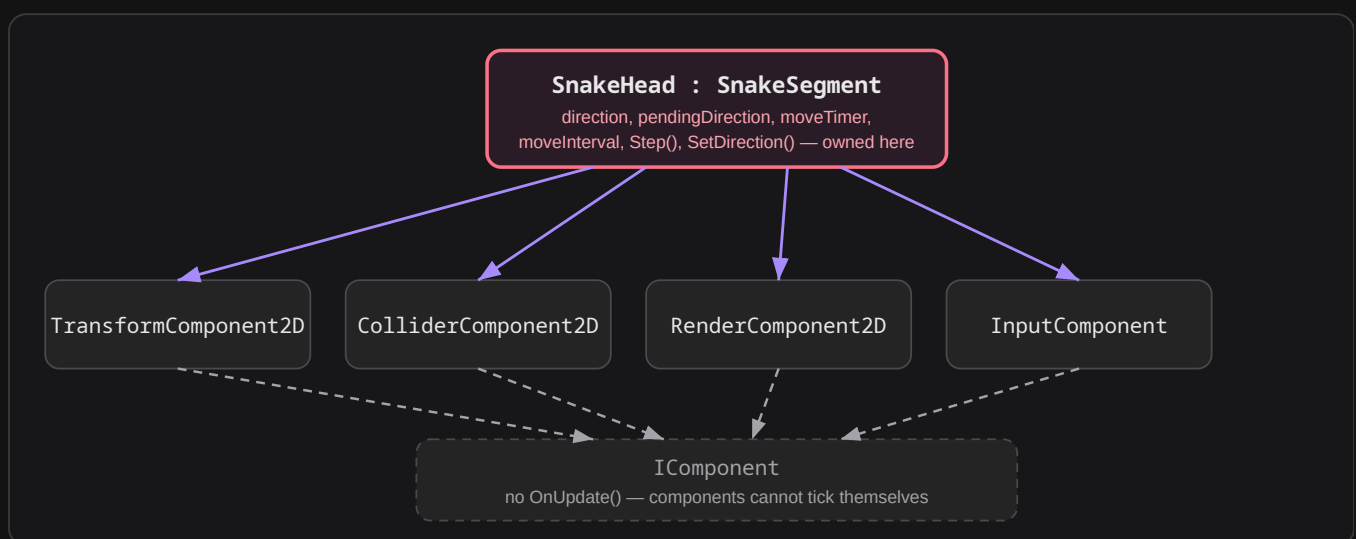
The proposed fix has three pieces, in order of foundational-ness:

1. **Components gain the ability to tick.** Today `IComponent` has no per-frame hook at all — `Entity` is the only thing the engine calls `Update()` on. Render and Input each worked around this with their own one-off manager (`RenderComponent2DManager`, `ActionRouter`), rather than a general capability.
2. A generic `MovementComponent` owns direction/timing state and its own input binding, and can be attached to *any* entity — the snake's head, or (as a test of the design) the apple.
3. The snake-specific "body follows the head" behavior stays out of `MovementComponent` entirely and instead reacts to a **data-carrying event** (`MovementSteppedEvent`) that `MovementComponent` publishes on the existing event bus — keeping the generic component generic, and the snake-specific behavior isolated to `SnakeBodyComponent`.

This document lays out the current state, the proposed shape, and a concrete adoption order for getting there.

Current state

`SnakeSegment` (and by inheritance `SnakeHead`) currently owns all of its own movement state directly, and `SnakeHead` wires player input straight into itself. Nothing about this is reusable — a second movable entity (the apple, an enemy, anything) would need the same fields and the same `Step()` logic copy-pasted onto it.



■ owns -.- extends (no per-frame hook) ■ movement state trapped on the entity, duplicated per-segment

Render works around the missing hook with its own manager walking a registered list

(`RenderComponent2DManager::OnRender`); Input works around it with `ActionRouter::Update` polling bound axes.

Both are one-off, component-specific answers to the same missing piece: **components have no generic way to run code once a frame.**

Proposed architecture

1. `IComponent` gains `OnUpdate(float deltaTime)`

Matches the pattern `IEntity` / `ISystem` already use: a virtual method with an empty default body (`virtual void OnUpdate(float) {}`), not pure — so existing components (`TransformComponent2D` , `ColliderComponent2D` , `RenderComponent2D` , `InputComponent`) need no changes at all, and only components that actually need a tick (like the new `MovementComponent`) override it. `Entity::Update()` iterates its own `components` map and calls `OnUpdate(deltaTime)` on each — one small, generic loop, replacing the need for any future one-off component manager.

Deliberately **out of scope for this pass**: giving components the rest of the entity lifecycle (`BeginPlay` / `OnActivate` / `OnRegistration`). Nothing needs it yet, and it's an easy follow-up if a real need shows up later.

2. `MovementComponent` — generic, reusable

Owens everything that used to live on `SnakeSegment` : `direction` , `pendingDirection` , `moveTimer` , `moveInterval` . Also owns its **own input binding** — it binds the "Up" / "Down" / "Left" / "Right" actions itself in its constructor (or an `OnRegistration` override), rather than the entity doing it. Attaching it to any entity makes that entity grid-step and respond to input, with zero entity-specific code:

```
// illustrative – not an applied edit
class MovementComponent : public Core::Components::IComponent {
public:
    MovementComponent(
        Core::Events::IEventBus& eventBus, Core::Input::ActionRouter& inputRouter,
        Core::Spatial::ITransform2D& transform, MovementComponentParams params
    );
    void OnUpdate(float deltaTime) override;
    void SetDirection(Core::Math::Vector2D value);

private:
    void Step();
    Core::Events::IEventBus& eventBus;
    Core::Spatial::ITransform2D& transform;
    Core::Math::Vector2D direction{};
    Core::Math::Vector2D pendingDirection{};
    float moveTimer = 0.0f;
    float moveInterval = 0.15f;
    float cellSize;
};
```

`SnakeHead` / `SnakeSegment` shed all of this — they go back to being plain transform + collider + render holders, exactly like the apple.

3. Body-following stays snake-specific, driven by a data-carrying event

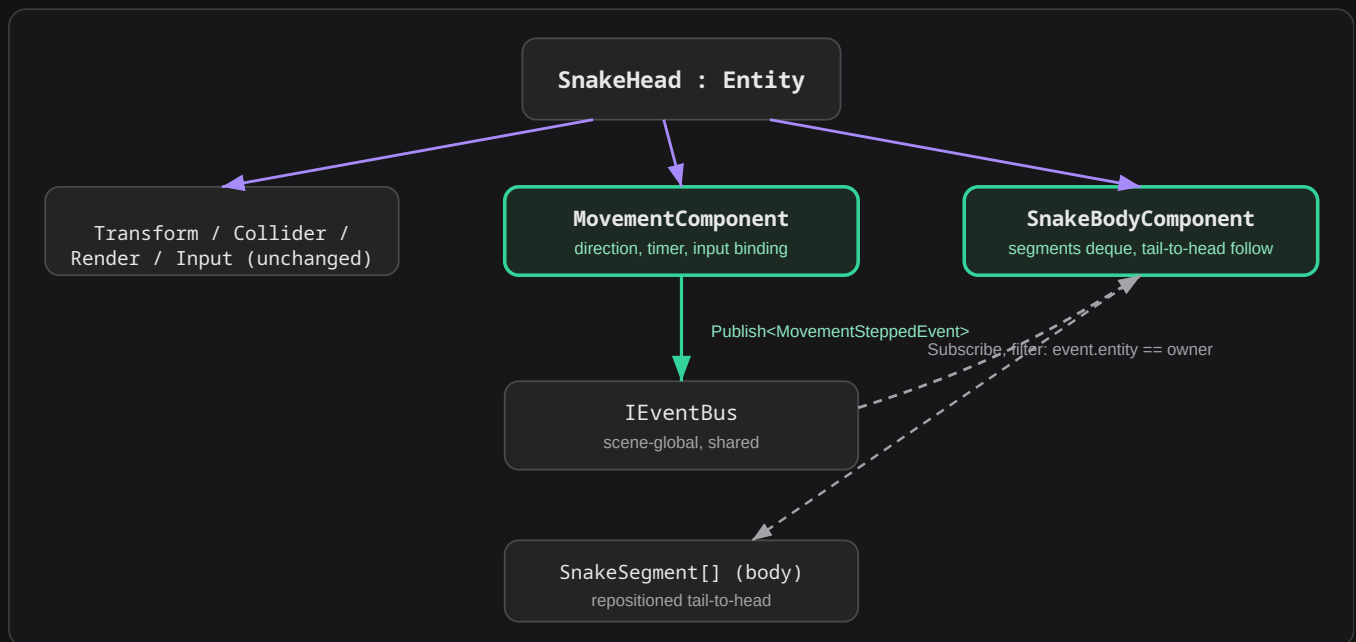
`MovementComponent` has no idea a "body" exists — attaching it to the apple must not drag a tail behind it. So the tail-to-head follow logic stays on `SnakeBodyComponent` (already snake-specific), and the two are decoupled through the **existing event bus**, which already supports arbitrary fields on events with zero changes needed —

`IEventBus::Publish<TEvent> / Subscribe<TEvent>` are fully generic over the concrete event type today (`i-event-bus.hpp:19-37`); `GameStartedEvent` / `AppleCollectedEvent` are only empty because nobody's put fields on one yet.

Because the event bus is scene-global (one shared bus for everything), the event has to carry enough to identify its source, so a subscriber can filter to "was that *my* head":

```
// illustrative
struct MovementSteppedEvent : Core::Events::IEvent {
    Core::Entities::IEntity* entity;
    Core::Math::Vector2D oldPosition;
    Core::Math::Vector2D newPosition;
};
```

`SnakeBodyComponent` subscribes, checks `event.entity == this->owner`, and on a match walks its segment deque **tail-to-head**, each segment taking the position of the one ahead of it *before* that one moves this tick — the standard "follow the leader" technique, and the reason the body traces the exact path the head took instead of cutting corners.



■ owns ■ publish --- subscribe / non-owning reaction

One grid tick, end to end

1. Player presses a direction key → `InputComponent` fires the bound digital action → `MovementComponent::SetDirection(...)` sets `pendingDirection`.
2. Every frame, `Entity::Update` ticks every component, including `MovementComponent`; its `OnUpdate` accumulates `moveTimer += deltaTime`.
3. Once `moveTimer >= moveInterval`: commit `pendingDirection` into `direction` (reversal-guarded), record the transform's old position, move it by `direction * cellSize`, then publish `MovementSteppedEvent{ this->owner, oldPos, newPos }`.

4. `SnakeBodyComponent::OnMovementStepped` checks `event.entity == this->owner`; if it matches, it walks its segment deque from the tail forward, each segment taking the position the one ahead of it held *before* this tick — finishing with the frontmost segment taking the head's `oldPosition`.

Adoption order

Step	Change	Why this order
1	Add <code>virtual void OnUpdate(float) {}</code> to <code>IComponent</code>	Foundational — nothing else compiles against a ticking component until this exists
2	<code>Entity::Update</code> iterates <code>components</code> and calls <code>OnUpdate</code> on each	Makes step 1 actually happen every frame
3	Add <code>MovementSteppedEvent</code> (entity + old/new position)	Needed before either side of the pub/sub pair can be written
4	Build <code>MovementComponent</code> : state, input binding, timer-gated <code>Step()</code> , publish on step	Self-contained — can be attached to the apple as a smoke test before touching the snake at all
5	Strip <code>direction</code> / <code>pendingDirection</code> / <code>moveTimer</code> / <code>moveInterval</code> / <code>Step()</code> / <code>SetDirection()</code> /input binding out of <code>SnakeSegment</code> / <code>SnakeHead</code> ; attach <code>MovementComponent</code> in <code>SnakeHead::OnRegistration</code> instead	Only safe once step 4 is proven working elsewhere
6	<code>SnakeBodyComponent</code> subscribes to <code>MovementSteppedEvent</code> , filters by entity, does the tail-to-head shift	Completes the feature — this is "Part 2" (body follows head)
7	(optional smoke test) Attach <code>MovementComponent</code> to <code>Apple</code> and confirm it grid-steps/responds to input with no apple-specific code	Validates the reusability goal directly, not just in theory

Summary

Goal from discussion	How this design addresses it
"Is it a problem that components can't tick?"	Yes — fixed generically via <code>IComponent::OnUpdate</code> , not a one-off for movement
"Movement should be a character feature, not a game system"	<code>MovementComponent</code> is attachable to any entity; no <code>GameSystem</code> involved
"If I gave the apple a movement component, it should just work"	Component has zero snake-specific knowledge; body-following lives entirely in <code>SnakeBodyComponent</code> instead
"We have an event system for this"	Reused as-is — <code>Publish / Subscribe</code> already support arbitrary event data, just needed fields added
"Could events carry callable behavior too?"	Deliberately out of scope — flagged as a separate, parallel idea (a request/command mechanism) to consider later, not conflated with broadcast events here